

Pulses and flows

State interventions at an instant, over an interval, and the invariants they must respect

State interventions come in two kinds. A *pulse* rewrites the state at an instant: vaccinate fifty people on day 15. A *flow* adds a process over a span: vaccinate at 2% per day for three weeks. Pulses act on `integrator.u` directly; flows are realised by *augmenting* the model with a new transition whose rate is a new parameter, so a flow becomes an ordinary parameter interval.

```
using CategoricalInterventions
using AlgebraicPetri
using DiffEqCallbacks
using LabelledArrays
using OrdinaryDiffEq
using Plots

default(; linewidth=2, grid=false)

sir = LabelledPetriNet([:S, :I, :R], :inf => ((:S, :I) => (:I, :I)), :rec => (:I => :R))
model = Model(sir)
u0 = LVector{S=990.0, I=10.0, R=0.0}
p0 = LVector{inf=0.05 * 10.0 / sum(u0), rec=0.25}
tspan = (0.0, 40.0)
```

(0.0, 40.0)

A pulse

A transfer is one atom. It removes from one state and adds to another, so the total is unchanged by construction.

```
pulse = @interventions model begin
    vaccination = transfer(:S => :R, 200.0; at=10.0)
end
pulse_sol = simulate(model, pulse; u0, p0, tspan, alg=Tsit5(), saveat=0.5)
(S_end=pulse_sol[1, end], R_end=pulse_sol[3, end], total=sum(pulse_sol.u[end]))
```

```
(S_end = 259.97448773805047, R_end = 728.6508526146367, total = 999.9999999999998)
```

A flow, by augmentation

```
campaign = @interventions model begin
    campaign = flow(:S => :R; rate=0.02, during=10.0..31.0)
end
aug = augment(model, campaign)
aug.model, aug.program.atoms
```

```
(Model{3 parameters, 3 states, continuous}, Atom[campaign: Parameter:flow_S_R_campaign set(0
```

The augmented net has a third transition from S to R, and the extended parameter vector carries its rate with baseline zero. Outside the span the augmented model is therefore the original model.

```
tnames(aug.model.metadata[:petri]), aug.extend(p0)
```

```
([:inf, :rec, :flow_S_R_campaign], 3-element LabelledArrays.LArray{Float64, 1, Vector{Float64}}
  :inf => 0.0005
  :rec => 0.25
  :flow_S_R_campaign => 0.0)
```

simulate performs the augmentation automatically when it sees a flow.

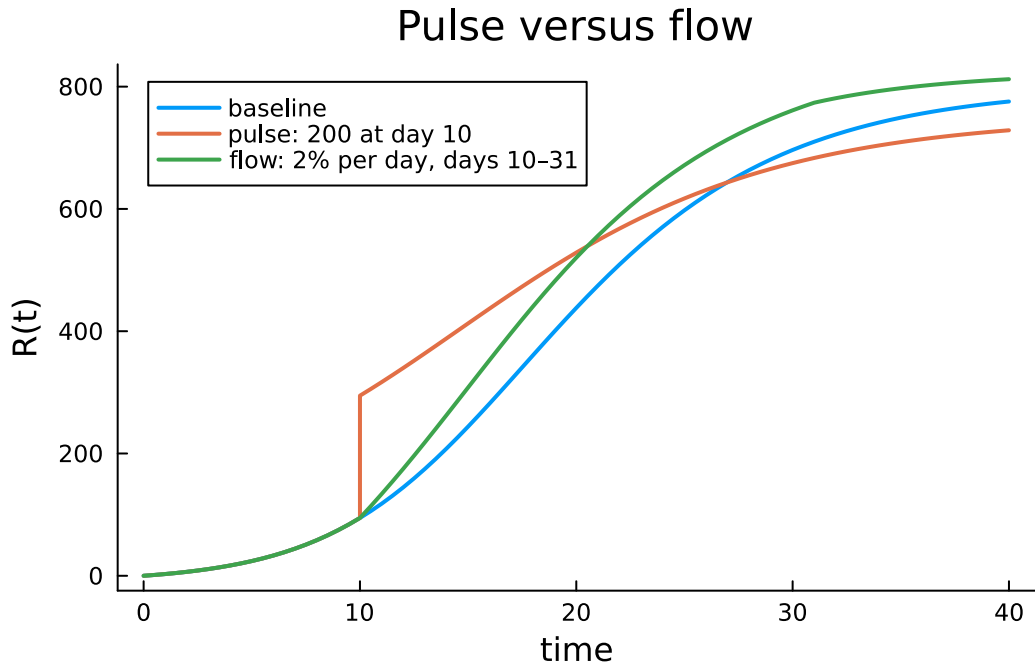
```
flow_sol = simulate(model, campaign; u0, p0, tspan, alg=Tsit5(), saveat=0.5)
baseline = simulate(model, Program(model); u0, p0, tspan, alg=Tsit5(), saveat=0.5)
(R_end=flow_sol[3, end], total=sum(flow_sol.u[end]))
```

```
(R_end = 812.1787786037709, total = 999.9999999999999)
```

```

plot(baseline.t, baseline[3, :]; label="baseline", xlabel="time", ylabel="R(t)", title="Pulse versus flow")
plot!(pulse_sol.t, pulse_sol[3, :]; label="pulse: 200 at day 10")
plot!(flow_sol.t, flow_sol[3, :]; label="flow: 2% per day, days 10-31")

```



Before the flow starts, the augmented model reproduces the baseline exactly.

```

early = simulate(model, campaign; u0, p0, tspan=(0.0, 10.0), alg=Tsit5(), saveat=0.5)
early_base = simulate(model, Program(model); u0, p0, tspan=(0.0, 10.0), alg=Tsit5(), saveat=0.5)
g = 0.0:0.5:10.0
maximum(abs, Array(early(g)) .- Array(early_base(g)))

```

0.0

Invariants

Because every transition of the net conserves tokens, the model was built with a conservation invariant. Pulses are checked against the model's invariants when they fire.

```

model.invariants

```

```
1-element Vector{CategoricalInterventions.AbstractInvariant}:
 Conserved([:S, :I, :R])
```

```
bad = @interventions model begin
    oops = add(:S, -2000.0; at=5.0)
end
try
    simulate(model, bad; u0, p0, tspan, alg=Tsit5())
catch err
    sprint(showerror, err)
end
```

```
"InvariantViolation: sum of [:S, :I, :R] changed from 1000.0000000000001 to -999.9999999999999"
```

A transfer conserves the total but can still drive a state negative; declare Nonnegative to catch that.

```
push!(model.invariants, Nonnegative([:S, :I, :R]))
too_many = @interventions model begin
    overdraw = transfer(:S => :R, 1500.0; at=5.0)
end
try
    simulate(model, too_many; u0, p0, tspan, alg=Tsit5())
catch err
    sprint(showerror, err)
end
```

```
"InvariantViolation: state S became negative (-556.537199256951)"
```

What the category theory bought

A transfer preserves the sum of the states (`transfer_preserves_sum`), which is why it is one atom rather than two paired additions. A flow becomes a parameter interval on the augmented model; adding atoms on a fresh target neither creates nor removes conflicts among the original atoms, and the original targets' schedules are unchanged (`augment_restrict`, `augment_conflictFree_iff`). The modelling assumption that a transition with rate zero does not change the vector field is stated, not proved.